Intervention	$N = 48$	$N = 64$	$N = 128$	$N = 256$	$N = 512$	$N = 1024$
Linear Unigram	0.00 ± 0.00	0.00 ± 0.00	0.01 ± 0.00	0.18 ± 0.03	0.91 ± 0.08	1.00 ± 0.00
Linear Bigram	0.01 ± 0.00	0.01 ± 0.00	0.54 ± 0.05	0.97 ± 0.05	1.00 ± 0.00	1.00 ± 0.00
Onion Unigram	0.83 ± 0.03	0.87 ± 0.03	0.89 ± 0.04	0.91 ± 0.08	0.95 ± 0.01	0.94 ± 0.04

Table 2: Intervention accuracy (mean of 5 runs; $\pm$ 1 s.d.) for GRUs of different sizes trained on the repeat task.

5.1 Interchange Intervention Data

In causal abstraction analysis (Geiger et al., 2021), interchange interventions are used to determine the content of a representation by fixing it to the counterfactual value it would have taken on if a different input were provided. These operations require datasets of counterfactuals. To create such examples, we begin with a random sequence **y** of length L consisting of elements of our vocabulary. We then sample a set of positions $I \subseteq \{1, \ldots, L\}$, where each position k has a 50% chance of being selected. To create the base **b**, we copy **y** and then replace each b_k with a random token, for $k \in I$. To create the source **s**, we copy **y** and then replace each s_j with a random token, for $j \notin I$. Here is a simple example with $I = \{1, 3\}$:

$$\mathbf{y} = \texttt{b d a c}$$
$$\mathbf{b} = \texttt{X d Y c}$$
$$\mathbf{s} = \texttt{b 4 a 1}$$

Our core question is whether we can replace representations obtained from processing **b** with those obtained from processing **s** in a way that leads the model to predict **y** in the decoding phase.

5.2 Method: Interchange Interventions on Unigram Subspaces

Our goal is to localize each position k in the input token sequence to a separate linear subspaces S_k of $\boldsymbol{h}_L$. We will evaluate our success using interchange interventions. For each position in $k \in I$, we replace the subspace S_k in the hidden representation $\boldsymbol{h}_L^{\mathbf{b}}$ for base input sequence **b** with the value it takes in $\boldsymbol{h}_L^{\mathbf{s}}$ for source input sequence **s**. The resulting output sequence should exactly match **y**. If we succeed, we have shown that the network has linear representations for each position in a sequence.

There is no reason to assume that the subspaces will be axis-aligned. Thus, we use Distributed Alignment Search (DAS) and train a rotation matrix $\boldsymbol{R} \in \mathbb{R}^{N \times N}$ to map $\boldsymbol{h}$ into a new rotated space $\bar{\boldsymbol{h}}$. However, a remaining difficulty is to determine which dimensions in the rotated space belong to which position. The size of individual subspaces may differ: for example, the first input of a repeated sequence, b_1, is always present, and the probability of successive inputs decreases due to the random length of the input sequences. Thus, the network might decide to allocate a larger subspace to the more important variables that are always present, maximizing the probability of correct decoding for popular sequence elements.

To solve this problem, we learn an assignment matrix $\boldsymbol{A} \in \{0, 1\}^{N \times (L+1)}$ that assigns dimensions of the axis-aligned representation $\bar{\boldsymbol{h}}$ with at most one sequence position. Allowing some dimensions to be unassigned provides the possibility for the network to store other information that is outside of these positions, such as the input length.

We can learn this assignment matrix by defining a soft version of it $\hat{\boldsymbol{A}} \in \mathbb{R}^{N \times (L+1)}$, and taking the hard gumbel-softmax (Jang et al., 2017; Maddison et al., 2017) with straight-through estimator (Hinton, 2012; Bengio et al., 2013) over its columns for each row ($r \in \{1 \ldots N\}$) independently:

$$\boldsymbol{A}[r] = \text{gumbel_softmax}(\hat{\boldsymbol{A}}[r]) \tag{5}$$

For intervening on the position $k \in \mathbb{N}$, we replace dimensions of the rotated state $\bar{\boldsymbol{h}}$, that are 1 in $\boldsymbol{A}[\cdot, v]$. Specifically, intervention $\hat{\boldsymbol{h}}^{\mathbf{b}}$ is defined:

$$\bar{\boldsymbol{h}}^{\mathbf{b}} = \boldsymbol{R}\boldsymbol{h}^{\mathbf{b}} \tag{6}$$
$$\bar{\boldsymbol{h}}^{\mathbf{s}} = \boldsymbol{R}\boldsymbol{h}^{\mathbf{s}} \tag{7}$$
$$\hat{\bar{\boldsymbol{h}}}^{\mathbf{b}} = \boldsymbol{A}[\cdot, v] \odot \bar{\boldsymbol{h}}^{\mathbf{s}} + (1 - \boldsymbol{A}[\cdot, v]) \odot \bar{\boldsymbol{h}}^{\mathbf{b}} \tag{8}$$
$$\hat{\boldsymbol{h}}^{\mathbf{b}} = \boldsymbol{R}^{\intercal}\hat{\bar{\boldsymbol{h}}}^{\mathbf{b}} \tag{9}$$

When learning the rotation matrix $\boldsymbol{R}$ and assignment matrix $\boldsymbol{A}$, we freeze the parameters of the already trained GRU network. We perform the intervention on the final state of the GRU, after encoding the input sequences, and use the original GRU to decode the output sequence $\hat{\mathbf{y}}$ from the intervened state $\hat{\boldsymbol{h}}_L^{\mathbf{b}}$. We update $\boldsymbol{R}$ and $\boldsymbol{A}$ by back-propogating with respect to the cross entropy loss between the output sequence $\hat{\mathbf{y}}$ and the expected output sequence after intervention **y**.

5.3 Results

We use the same training set as the base model to train the intervention model, and we use the same validation set to evaluate it. The first row of Table 2 shows the accuracy of the unigram intervention. It works well for "big" models, with $N \geq 512$. In these cases, we can confidentially conclude that the model has a separate linear subspace for each position in the sequence.

5.4 Discussion

The above results suggest that the model prefers to store each input element in a different subspace if there is "enough space" in its representations relative to the task. However, Hypothesis 1 seems to be incorrect for autoregressive decoders where $N < 512$. Since these models do solve our task, we need to find an alternative explanation for how they succeed. This leads us to Hypothesis 2.

6 Hypothesis 2: Bigram Variables

Our second hypothesis is a minor variant of Hypothesis 1. Here, we posit that, instead of representing variables for unigrams, the model instead stores tuples of inputs (i_t, i_{t+1}) we call bigram variables.

6.1 Intervention Data

We create counterfactual pairs using the same method as we used for Hypothesis 1 (section 5.1). In this case, each token i_t affects two bigram variables (if present). Thus, the subspace replacement intervention must be performed on both of these variables. This also means that, for each $k \in I$, the tokens s_{k-1} and s_{k+1} in the source sequence input must match b_{t-1} and b_{t+1} in the base sequence, because the bigram at position $t-1$ depends on (i_{t-1}, i_t) and the bigram at t depends on (i_t, i_{t+1}).

6.2 Method: Interchange Interventions on Bigram Subspaces

For a sequence of length L, there are $L-1$ bigram variables. To try to identify these, we use the same interchange intervention method described in section 5.2. Because targeting a single position in the base input sequence requires replacing two bigram variables, we intervene on only a single token at a time. Otherwise, the randomized sequence could be too close to the original, and most of the subspaces would be replaced at once, thereby artificially simplifying the task.

6.3 Results

We show the effectiveness of bigram interventions in the middle row of Table 2. The intervention is successful on most sizes, but fails for the smallest models ($N \leq 64$).

6.4 Discussion

We hypothesize that the models prefer to learn bigram representations because of their benefits for autoregressive input: the current input can be compared to each of the stored tuples, and the output can be generated from the second element of the tuple. This alone would be enough to repeat all sequences which have no repeated tokens. Because our models solve the task with repeat tokens, an additional mechanism must be involved. Regardless, bigrams could provide a powerful representation that is advantageous for the model.

Two additional remarks are in order. First, successful unigram interventions entail successful bigram interventions; a full argument is given in Appendix E.1. Second, one might worry that our negative results for smaller models trace to limitations of DAS on the small models. Appendix E.2 addresses this by showing DAS succeeding on a non-autoregressive control model ($N \leq 64$) that solves the copy task. This alleviates the concern, suggesting that the small autoregressive model does not implement the bigram solution and highlighting the role of autoregression in the bigram solution.

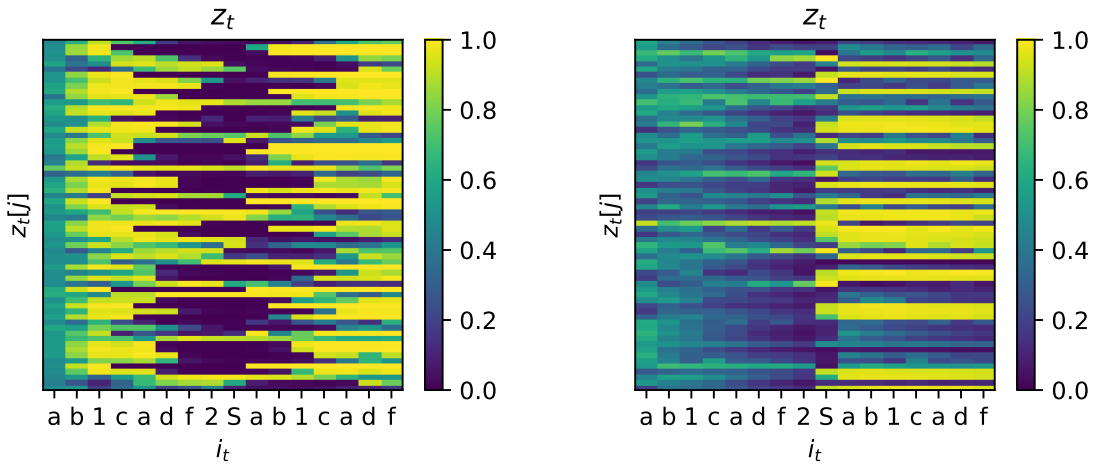
However, we still do not have an explanation for how the smallest models ($N \leq 64$) manages to solve the repeat task; Hypotheses 1 and 2 are unsupported as explanations for this model. This in turn leads us to Hypothesis 3.

7 Hypothesis 3: Onion Representations

In an effort to better understand how the smallest GRUs solve the repeat task, we inspected the gate values $\boldsymbol{z}_t$ as defined in equation 1 from the GRU definition (section 3).

Figure 2a visualizes the first 64 input gates for the $N = 1024$ model (Appendix figure 5 is a larger diagram with all the gates). The x-axis is the sequence (temporal dimension) and the y-axis depicts the gate for each dimension. One can see that this model uses gates to store inputs by closing position-dependent channels sharply, creating a position-dependent subspace for each input. (This gating pattern is consistent across all inputs.)

Figure 2b shows all the gates for the $N = 64$



(a) The first 64 channels of GRU with $N = 1024$. The model learns to store variables in different, axis-aligned subspaces. Gates close sharply, freezing individual subspaces at different times. For all channels, please refer to Figure 5 in the Appendix.

(b) GRU with $N = 64$ learns a "onion representation", using different scales of the same numbers to represent the variables. The gates close gradually and synchronously in the input phase, providing the exponentially decaying scaling needed to represent different positions in the sequence.

Figure 2: The input gate $\boldsymbol{z}_t$ in GRUs learning different representations Yellow is open; dark blue is closed; y-axis is the channel; x axis is the position. Both models use input gates to let in different proportions of each dimension across the sequence in order to store the positions of the input tokens. The large model (left) sharply turns off individual channels to mark position; in contrast, the small model (right) gradually turns off all channels.

model. Here, the picture looks substantially different. This model gradually closes its gates simultaneously, suggesting that the network might be using this gate to encode token positions. This led us to Hypothesis 3: RNNs learn to encode each position in a sequence as a magnitude.

This hypothesis relies heavily on the autoregressive nature of the GRU, the discriminative capacity of the output classifier $g(\boldsymbol{h}_t)$, and the sequential nature of the problem. Multiple features can be stored in the same subspace, at different scales. When the GRU begins to generate tokens at timestep $t = L + 2$, if the scales $s_{t'}$ associated with position $t' > t$ are sufficiently small ($s_{t'} \ll s_t$), the output classifier $y_t = g(\boldsymbol{h}_t)$ will be able to correctly decode the first input token i_1. In the following step, i_1 is fed back to the model as an input, and the model is able to remove the scaled representation corresponding to i_1 from $\boldsymbol{h}_t$, obtaining $\boldsymbol{h}_{t+1}$. In this new representation, the input with the next largest scale, i_2, will be dominant and will be decoded in the next step. This can be repeated to store a potentially long sequence in the same subspace, limited by the numerical precision. We call these 'onion representations' to invoke peeling back layers corresponding to sequence positions.

Hypothesis 3 falls outside of the LRH. In linear representations, tokens are directions and each position has its own subspace. All positions are independently accessible; tokens can be read-out and manipulated given the right target subspace. Onion representations have very different characteristics.

First, tokens have the same direction regardless of which position they are stored in; the magnitude of the token embedding determines the position rather than its direction. As a result, if multiple positions contain the same token, the same direction will be added twice with different scaling factors (see figure 1d where the token c occurs in positions 2 and 3). Second, because the memory is the sum of the scaled token embeddings, it is impossible to isolate the position associated with a given scale. Only the token with the most dominant scale can be extracted at a given time, by matching it to a dictionary of possible token directions. This is done by the final classifier for our GRUs. The autoregressive feedback for GRUs in effect peels off each layer, clearing access to the next variable.

Appendix F provides a toy implementation of the onion solution to elucidate the underlying concepts.

7.1 Intervention Data

For the causal analysis of onion representations, we do not use interchange interventions. Instead, we learn an embedding matrix for each token that